

DOCUMENTACIÓN API REST - PRODUCTOS

1. Introducción

Este documento describe el funcionamiento de una API REST desarrollada en PHP que permite gestionar productos almacenados en una base de datos MySQL.

La API devuelve y recibe información en formato JSON y está diseñada para ser reutilizable por cualquier frontend compatible con HTTP.

2. Tecnologías utilizadas

- Servidor web: Apache (XAMPP)
- Lenguaje de servidor: PHP
- Base de datos: MySQL
- Acceso a datos: PDO
- Formato de datos: JSON
- Protocolo: HTTP
- Cliente: HTML + JavaScript (fetch)

3. Base de datos

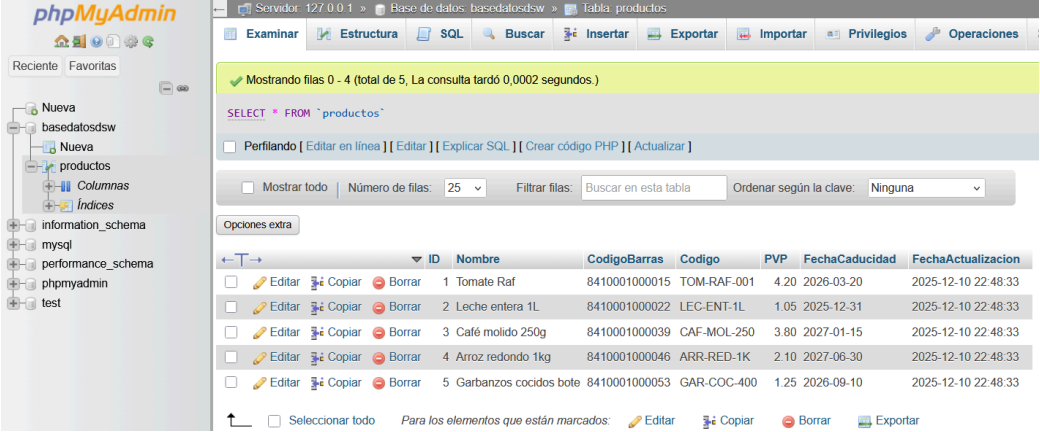
Base de datos: basedatosdsw

Tabla: productos

Campos principales:

- ID (clave primaria)
- Nombre
- CodigoBarras
- Codigo
- PVP
- FechaCaducidad
- FechaActualizacion

En esta captura se muestra la base de datos basedatosdsw con la tabla productos, donde se almacenan los datos gestionados por el servicio web.



Mostrando filas 0 - 4 (total de 5, La consulta tardó 0,0002 segundos)

SELECT * FROM `productos`

Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]

Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna

Opciones extra

	ID	Nombre	CodigoBarras	Codigo	PVP	FechaCaducidad	FechaActualizacion
☐	1	Tomate Raf	8410001000015	TOM-RAF-001	4.20	2026-03-20	2025-12-10 22:48:33
☐	2	Leche entera 1L	8410001000022	LEC-ENT-1L	1.05	2025-12-31	2025-12-10 22:48:33
☐	3	Café molido 250g	8410001000039	CAF-MOL-250	3.80	2027-01-15	2025-12-10 22:48:33
☐	4	Arroz redondo 1kg	8410001000046	ARR-RED-1K	2.10	2027-06-30	2025-12-10 22:48:33
☐	5	Garbanzos cocidos bote	8410001000053	GAR-COC-400	1.25	2026-09-10	2025-12-10 22:48:33

Seleccionar todo | Para los elementos que están marcados: Editar Copiar Borrar Exportar

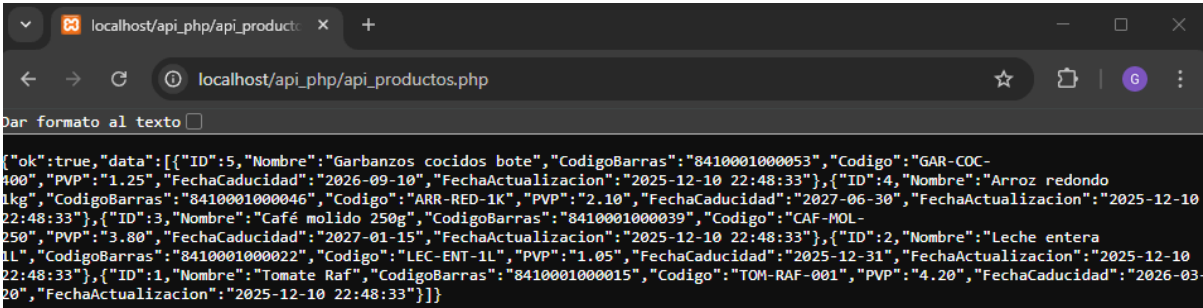
4. Endpoint principal

URL base:

http://localhost/api_php/api_productos.php

Todas las operaciones se realizan sobre este endpoint usando distintos métodos HTTP.

Respuesta del endpoint api_productos.php devolviendo el listado de productos en formato JSON mediante una petición HTTP GET.



localhost/api_php/api_productos.php

Dar formato al texto

```
{
  "ok": true,
  "data": [
    {
      "ID": 5,
      "Nombre": "Garbanzos cocidos bote",
      "CodigoBarras": "8410001000053",
      "Codigo": "GAR-COC-400",
      "PVP": "1.25",
      "FechaCaducidad": "2026-09-10",
      "FechaActualizacion": "2025-12-10 22:48:33"
    },
    {
      "ID": 4,
      "Nombre": "Arroz redondo 1kg",
      "CodigoBarras": "8410001000046",
      "Codigo": "ARR-RED-1K",
      "PVP": "2.10",
      "FechaCaducidad": "2027-06-30",
      "FechaActualizacion": "2025-12-10 22:48:33"
    },
    {
      "ID": 3,
      "Nombre": "Café molido 250g",
      "CodigoBarras": "8410001000039",
      "Codigo": "CAF-MOL-250",
      "PVP": "3.80",
      "FechaCaducidad": "2027-01-15",
      "FechaActualizacion": "2025-12-10 22:48:33"
    },
    {
      "ID": 2,
      "Nombre": "Leche entera 1L",
      "CodigoBarras": "8410001000022",
      "Codigo": "LEC-ENT-1L",
      "PVP": "1.05",
      "FechaCaducidad": "2025-12-31",
      "FechaActualizacion": "2025-12-10 22:48:33"
    },
    {
      "ID": 1,
      "Nombre": "Tomate Raf",
      "CodigoBarras": "8410001000015",
      "Codigo": "TOM-RAF-001",
      "PVP": "4.20",
      "FechaCaducidad": "2026-03-20",
      "FechaActualizacion": "2025-12-10 22:48:33"
    }
  ]
}
```

5. Operaciones disponibles (CRUD)

5.1 Obtener listado de productos

Método: GET

URL:

/api_productos.php

Respuesta correcta (200):

```
{  
  "ok": true,  
  "data": [ ... ]  
}
```

5.2 Obtener producto por ID

Método: GET

URL:

/api_productos.php?id=3

Errores posibles:

404 → producto no encontrado

5.3 Crear producto

Método: POST

Body (JSON):

```
{  
  "Nombre": "Pan",  
  "PVP": 1.25  
}
```

Respuesta correcta (201):

```
{  
  "ok": true,  
  "message": "Producto creado",  
  "id": 6  
}
```

5.4 Actualizar producto

Método: PUT

Body (JSON):

```
{
  "ID": 3,
  "Nombre": "Pan integral",
  "PVP": 1.40
}
```

Errores posibles:

- 400 → faltan campos
- 404 → producto no encontrado

5.5 Eliminar producto

Método: DELETE

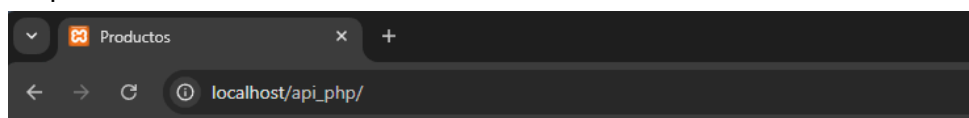
URL:

/api_productos.php?id=3

Respuesta correcta (200):

```
{
  "ok": true,
  "message": "Producto borrado"
}
```

Frontend desarrollado en HTML y JavaScript que consume la API REST mediante fetch() y muestra los productos obtenidos del servidor.



Listado de productos

✅ Productos cargados

ID	Nombre	Código barras	Código interno	PVP	Fecha caducidad	Actualizado
5	Garbanzos cocidos bote	8410001000053	GAR-COC-400	1.25	2026-09-10	2025-12-10 22:48:33
4	Arroz redondo 1kg	8410001000046	ARR-RED-1K	2.10	2027-06-30	2025-12-10 22:48:33
3	Café molido 250g	8410001000039	CAF-MOL-250	3.80	2027-01-15	2025-12-10 22:48:33
2	Leche entera 1L	8410001000022	LEC-ENT-1L	1.05	2025-12-31	2025-12-10 22:48:33
1	Tomate Raf	8410001000015	TOM-RAF-001	4.20	2026-03-20	2025-12-10 22:48:33

6. Respuestas de error (Error responses)

La API devuelve siempre las respuestas de error en formato JSON y con un código HTTP adecuado.

Esto permite que cualquier cliente (frontend u otro servicio) pueda interpretar correctamente el fallo y actuar en consecuencia.

6.1 Formato estándar de error

Todas las respuestas de error siguen la misma estructura:

```
{
  "ok": false,
  "code": "DB_ERROR",
  "error": "No hay conexión con la base de datos"
}
```

Descripción de campos:

- **ok**: indica si la operación se ha realizado correctamente (true / false)
- **code**: código interno de error, estable y pensado para ser interpretado por el frontend
- **error**: mensaje descriptivo legible para el usuario o desarrollador

6.2 Códigos de error del backend (API REST)

La siguiente tabla recoge todos los errores que puede devolver la API desde el servidor, indicando el código HTTP, el código interno y la situación en la que se produce.

Situación	Código HTTP	Code	Descripción	Ejemplo
JSON inválido en el body	400	BAD_JSON	El cuerpo de la petición no contiene un JSON válido	POST / PUT con JSON mal formado
Faltan campos obligatorios	400	MISSING_FIELDS	No se envían los campos requeridos (Nombre, PVP)	POST /api_productos.php
Falta ID	400	MISSING_ID	No se proporciona el ID para actualizar o borrar	PUT / DELETE sin ID
Producto no encontrado	404	NOT_FOUND	El producto solicitado no existe	GET /api_productos.php?id=999
Método no permitido	405	METHOD_NOT_ALLOWED	Se usa un método HTTP no soportado	PATCH /api_productos.php
Error de base de datos	500	DB_ERROR	No se puede conectar con la base de datos	MySQL apagado
Error interno del servidor	500	SERVER_ERROR	Error inesperado en el servidor	Excepción no controlada

6.3 Ejemplos de respuestas de error

Error de conexión con la base de datos

```
{
  "ok": false,
  "error": "No hay conexión con la base de datos",
  "code": "DB_ERROR"
}
```

Producto no encontrado

```
{
  "ok": false,
  "error": "Producto no encontrado",
  "code": "NOT_FOUND"
}
```

Campos obligatorios no enviados

```
{
  "ok": false,
  "error": "Faltan campos obligatorios: Nombre y PVP",
  "code": "MISSING_FIELDS"
}
```

6.4 Errores detectados en el cliente (Frontend)

Además de los errores devueltos por la API, el frontend puede detectar errores propios durante el consumo del servicio mediante `fetch()`.

Estos errores no proceden del servidor, sino del cliente.

Situación	Tipo	Code	Descripción	Cómo reproducir
Timeout del cliente	Cliente	TIMEOUT	La petición se cancela por exceder el tiempo máximo	Timeout muy bajo (ej. 50 ms)
Error de red / servidor caído	Cliente	FETCH_FAILED	No se recibe ninguna respuesta HTTP	Servidor apagado o puerto incorrecto
Respuesta no JSON	Cliente	NON_JSON	El servidor devuelve HTML en lugar de JSON	Endpoint inexistente

6.5 Pruebas realizadas

Para comprobar el correcto funcionamiento del control de errores se han realizado las siguientes pruebas:

- Peticiones correctas con respuesta HTTP 200
- Peticiones a endpoints inexistentes (HTTP 404)

- Apagado del servicio MySQL para provocar error HTTP 500
- Pruebas de estrés simulando timeouts y errores de red

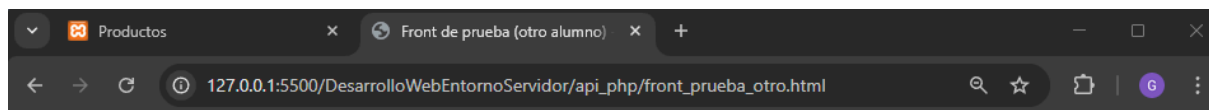
Ejemplo de prueba mediante curl:

```
curl.exe -i "http://localhost/api_php/api_productos.php"
```

6.6 Gestión de errores en el servidor

La API utiliza bloques try-catch y convierte los avisos y errores de PHP en excepciones, garantizando que cualquier fallo interno sea devuelto siempre en formato JSON y no como HTML del servidor.

Esto asegura una respuesta consistente y facilita el tratamiento de errores por parte del frontend.



Front de prueba — API Productos

Esta página está pensada para simular el "front de otro alumno".

URL del endpoint

GET / productos

Probar CORS

✖ GET falló: Respuesta no-JSON

POST — Crear producto

Nombre *

PVP *

Código de barras

Código

Fecha caducidad (YYYY-MM-DD)

Crear (POST)

PUT — Actualizar producto

ID *

Nombre *

PVP *

Código de barras

Código

Fecha caducidad (YYYY-MM-DD)

Actualizar (PUT)

DELETE — Borrar producto

ID *

Borrar (DELETE)

Nota: aquí usamos ?id=... en la URL, que es lo más simple.

GET — Producto por ID

ID *

GET por ID

Respuesta

```
{
  "ok": false,
  "error": "Respuesta no-JSON",
  "code": "NON_JSON"
}
```

7. Control de acceso (CORS)

La API incluye configuración CORS para permitir el acceso desde otros frontends ubicados en distintos puertos u orígenes.

Esto garantiza que el servicio puede ser reutilizado por aplicaciones externas sin bloqueos del navegador.

Se ha verificado el correcto funcionamiento mediante un frontend de prueba externo y el botón “Probar CORS”.

“El botón ‘Probar CORS’ sirve para comprobar que la API puede ser consumida desde otros frontends en distintos puertos u orígenes, algo necesario para que el servicio sea reutilizable.”

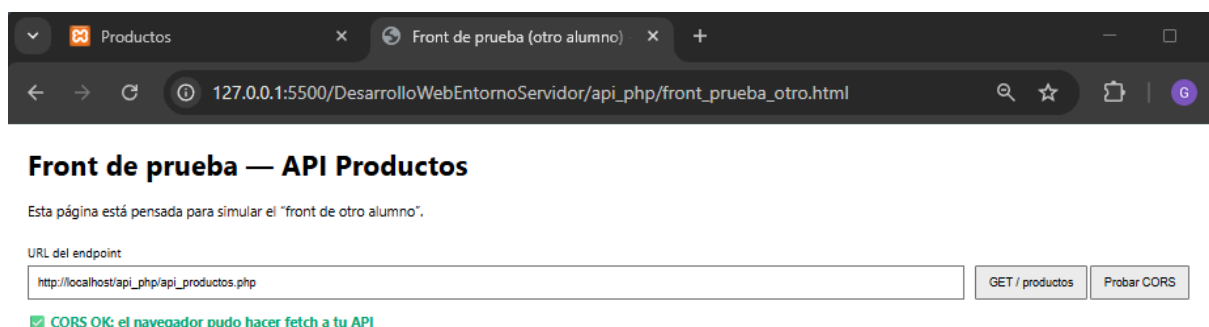
Cuando pulsas el botón:

1. El navegador intenta hacer un `fetch()` a la API.
2. Antes de permitir la respuesta, el navegador comprueba si el servidor **autoriza accesos externos**.
3. La API responde con estos headers:

Access-Control-Allow-Origin: *

Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
4. El navegador entiende:

“Este servidor permite que cualquier frontend acceda”
5. La petición se completa correctamente y el botón muestra **CORS OK**.



Frontend independiente que simula el uso del servicio web por parte de otro alumno, comprobando que la API puede ser consumida desde otro cliente distinto.

Productos

Front de prueba (otro alumno)

127.0.0.1:5500/DesarrolloWebEntornoServidor/api_php/front_prueba_otro.html

Front de prueba — API Productos

Esta página está pensada para simular el "front de otro alumno".

URL del endpoint

http://localhost/api_php/api_productos.php

GET / productos

Probar CORS

✓ CORS OK: el navegador pudo hacer fetch a tu API

POST — Crear producto

Nombre *

Ej: Pan

PVP *

Ej: 1.25

Código de barras

Ej: 841...

Código

Ej: PAN-001

Fecha caducidad (YYYY-MM-DD)

2026-05-01

Crear (POST)

PUT — Actualizar producto

ID *

Ej: 3

Nombre *

Ej: Pan integral

PVP *

Ej: 1.40

Código de barras

Ej: 841...

Código

Ej: PAN-002

Fecha caducidad (YYYY-MM-DD)

2026-05-10

Actualizar (PUT)

DELETE — Borrar producto

ID *

Ej: 3

Borrar (DELETE) Nota: aquí usamos ?id=... en la URL, que es lo más simple.

GET — Producto por ID

ID *

Ej: 1

GET por ID

Respuesta

```
{
  "ok": true,
  "data": [
    {
      "ID": 5,
      "Nombre": "Garbanzos cocidos bote",
      "CodigoBarras": "8410001000053",
      "Codigo": "GAR-COC-400",
      "PVP": "1.25",
      "FechaCaducidad": "2026-09-10",
      "FechaActualizacion": "2025-12-10 22:48:33"
    },
    {
      "ID": 4,
      "Nombre": "Arroz redondo 1kg",
      "CodigoBarras": "8410001000046",
      "Codigo": "ARR-RED-1K",
      "PVP": "2.10",
      "FechaCaducidad": "2027-06-30",
      "FechaActualizacion": "2025-12-10 22:48:33"
    }
  ]
}
```

8. Conclusión

La API desarrollada es funcional, reutilizable y accesible mediante HTTP. Permite gestionar productos de forma segura y controlada, cumpliendo los requisitos del módulo de Entorno Servidor.